

2. Prove that the class of decidable languages is closed under complementation, i.e. if L is any decidable language, then so is $\Sigma^* - L$.

Proof

1)

- Let L be a decidable language
- This means there exists a Turing Machine M that decides L
- M always halts and outputs either "accept" or "reject" for any input w
- If $w \in L$, then M accepts w
- If $w \notin L$, then M rejects w

2) To prove $\Sigma^* - L$ is decidable, we need to construct a Turing Machine M' that decides $\Sigma^* - L$

3) We can construct M' as follows

- M' will run exactly like M on any input
- However, M' will "flip" the output of M :
 - * When M would accept, M' will reject
 - * When M would reject, M' will accept

4) Now we need to prove M' decides $\Sigma^* - L$:

a) First, let's prove M' halts on all inputs:

- Since L is decidable, M halts on all inputs
- M' just runs M and flips the output
- Therefore, M' also halts on all inputs

b) Next, let's prove M' correctly decides $\Sigma^* - L$:

- For any $w \in \Sigma^* - L$:
 - * This means $w \notin L$
 - * Therefore, M rejects w
 - * Thus, M' accepts w
- For any $w \notin (\Sigma^* - L)$:
 - * This means $w \in L$
 - * Therefore, M accepts w
 - * Thus, M' rejects w

5)

- M' decides $\Sigma^* - L$
- M' always halts
- M' correctly accepts all strings in $\Sigma^* - L$ and rejects all strings not in $\Sigma^* - L$
- Therefore, $\Sigma^* - L$ is decidable

Thus, we have proven that if L is decidable, then $\Sigma^* - L$ is also decidable, showing that the class of decidable languages is closed under complementation.

3. Prove that the class of Turing-recognizable languages is closed under union, i.e. if L_1 and L_2 are any Turing-recognizable languages, then so is $L_1 \cup L_2$.

Proof:

- 1)
 - Let L_1 and L_2 be Turing-recognizable languages
 - This means there exist Turing Machines M_1 and M_2 that recognize L_1 and L_2 respectively
 - If $w \in L_1$, then M_1 accepts w (halts in accept state)
 - If $w \in L_2$, then M_2 accepts w (halts in accept state)
 - If $w \notin L_1$, then M_1 either rejects w or loops forever
 - If $w \notin L_2$, then M_2 either rejects w or loops forever
- 2) To prove $L_1 \cup L_2$ is Turing-recognizable, we need to construct a Turing Machine M that recognizes $L_1 \cup L_2$
- 3) The key idea is to run M_1 and M_2 "in parallel" on the input:
 - We will simulate running both machines simultaneously
 - If either machine accepts, we accept
 - This ensures we don't get stuck if one machine loops while the other would accept
- 4) Now we need to prove M recognizes $L_1 \cup L_2$:
 - a) First, let's prove M accepts all strings in $L_1 \cup L_2$:
 - Let $w \in L_1 \cup L_2$
 - Then either $w \in L_1$ or $w \in L_2$ (or both)
 - If $w \in L_1$, then M_1 will eventually accept w
 - If $w \in L_2$, then M_2 will eventually accept w
 - In either case, M will detect this acceptance and accept w
 - b) Next, let's prove M doesn't accept any strings not in $L_1 \cup L_2$:
 - Let $w \notin L_1 \cup L_2$
 - This means $w \notin L_1$ and $w \notin L_2$
 - Therefore, neither M_1 nor M_2 will accept w
 - Thus, M will not accept w
- 5) The parallel simulation can be achieved by:
 - Using multiple tapes (Turing machines can have multiple tapes)
 - Keeping track of the configurations of both M_1 and M_2
 - Alternating between simulating one step of each machine
- 6)
 - M accepts exactly the strings in $L_1 \cup L_2$
 - If a string is in $L_1 \cup L_2$, M will accept it
 - If a string is not in $L_1 \cup L_2$, M will not accept it
 - Therefore, $L_1 \cup L_2$ is Turing-recognizable

Thus, we have proven that if L_1 and L_2 are Turing-recognizable, then $L_1 \cup L_2$ is also Turing-recognizable, showing that the class of Turing-recognizable languages is closed under union.